GazePose-Context: Multimodal Context Inference for Adaptive XR Training in Unity and Meta Quest

Lavanya D. Varun Siddaraju Dr. Alexandra Kitson

Abstract

Adaptive extended reality (XR) systems need to respond to user attention, disengagement, and task flow without requiring explicit user input. This paper presents GazePose-Context, a Unity-based XR prototype that infers user context during a simple training task by combining gaze, head pose, body posture, hand interaction, and spatial context signals. The system organizes processing into three layers: sensor abstraction, context inference, and XR adaptation. Multimodal signals are synchronized into a shared SignalFrame, transformed into modality-specific feature vectors, fused through interpretable rules, and stabilized with a hysteresis-based state machine to estimate four context states: Engaged, Distracted, Transitioning, and Idle. These states drive lightweight adaptive behaviors such as subtle focus reinforcement, redirection prompts, next-step support, and rest-oriented controls. The current implementation targets Meta Quest deployment using Unity 2022.3.62f1, OpenXR 1.14.3, Meta XR All-in-One SDK 201.0.0, and OVRPlugin 1.201.0. This paper reports the current implementation status in the repository, integrates the reference figures from the earlier draft, and prepares the manuscript in paper-ready LaTeX format while leaving explicit placeholders only where verified literature and user-study results are still missing.

Keywords: XR, adaptive interfaces, multimodal interaction, gaze, body pose, hand tracking, context inference, Meta Quest

1 Introduction

Immersive systems are increasingly expected to understand what a user is doing and to react in ways that support attention, task completion, and comfort. In XR training environments, these expectations are especially strong because users often move between focus, transition, confusion, interruption, and rest without explicitly announcing those states. A system that can infer context from ongoing behavior can potentially guide the user without interrupting task flow or overloading the interface.

Most current XR experiences still rely on explicit UI actions, hard-coded task triggers, or isolated interaction signals. A single modality rarely captures the full structure of user behavior. Gaze alone cannot distinguish between task engagement and exploratory scanning. Hand interaction alone cannot represent attention drift. Body posture or locomotion alone can be ambiguous without knowing where the user is looking or whether they are interacting with task objects. This motivates a multimodal approach to context inference.

This paper presents **GazePose-Context**, an XR prototype developed in Unity for Meta Quest-class devices. The prototype combines gaze-related cues, head pose, posture and joint movement, hand pinch interactions, and spatial boundary context to infer one of four user states: *Engaged*, *Distracted*, *Transitioning*, and *Idle*. These states are then mapped to lightweight adaptive behaviors inside a simple training simulation. The task environment is intentionally constrained: the user sorts three virtual objects into matching receptacles. This simple task reduces gameplay complexity while making attention, object interaction, and task transitions easier to observe and interpret.

The current paper reflects the actual repository status rather than the earlier unfinished draft. The implementation side is now substantially stronger: the repository contains the signal schema, synchronization, feature extraction, rule-based fusion, state hysteresis, XR adaptation, and logging pipeline. The empirical evaluation side remains partially pending because the final participant study results and the full verified bibliography were not recoverable from the local draft assets.

The contributions of this work are:

1. A three-layer XR architecture for multimodal context inference and adaptive support.
2. A synchronized runtime signal representation combining gaze, head, body, hand, and spatial context data.
3. An interpretable rule-based context engine that estimates Engaged, Distracted, Transitioning, and Idle states with confidence scores.
4. A boundary-aware adaptation layer that changes feedback strategy based on user state and spatial setup.
5. A logging and evaluation design aligned with a within-subjects adaptive-vs-baseline user study.

2 Related Work

The earlier draft files available locally did not preserve a full machine-readable bibliography. As a result, this LaTeX version restores only the citations that can be verified directly from the surviving reference notes and current implementation. The remaining related-work entries should be inserted from Lavanya's original literature survey once the source list is available.

2.1 Gaze and Attention Modeling

The current implementation uses an Identification by Velocity Threshold (I-VT) style fixation detector with a 30 degrees/second velocity threshold and an 80 ms minimum fixation duration. These values align with a standard fixation-identification approach in eye-tracking literature [1]. The use of dwell ratio and saccade rate as secondary features follows the same general logic of aggregating gaze dynamics into interpretable behavioral features.

2.2 Workload and User Evaluation

The evaluation plan uses NASA-TLX as a secondary measure of subjective workload, which is standard for human factors and interactive systems evaluation [2]. In the planned study, NASA-TLX is intended to complement behavioral metrics such as context detection accuracy, task completion time, and sorting errors.

2.3 Pending Reference Integration

The earlier draft clearly intended a larger related-work section covering adaptive XR systems, multimodal context inference, and training-oriented immersive interfaces. Those additional references were not recoverable in full from the local draft assets. This paper therefore keeps the related-work framing compact and implementation-grounded for now, and should be expanded once the original reference list is restored.

3 Problem Statement and Research Questions

The core problem addressed in this paper is how to infer task-relevant user context in real time during immersive interaction and use that inference to adapt XR behavior without requiring manual status input from the user.

The primary research question is:

RQ1. Can multimodal XR signals be fused in real time to infer task-relevant user context during a simple immersive training activity?

The secondary research question is:

RQ2. Does context-aware adaptive support improve task performance and subjective experience relative to a non-adaptive baseline?

These questions are operationalized through a sorting task in which the user places three objects into corresponding receptacles. The task provides a controlled environment in which gaze targets, hand interactions, posture changes, and task transitions can be observed with relatively low confound from gameplay complexity.

4 System Architecture

The implemented prototype follows a three-layer architecture:

1. **Sensor Abstraction Layer**
2. **Context Engine**
3. **XR App Shell**

This architecture mirrors the planning documents and is also reflected in the current repository structure.

4.1 Layer 1: Sensor Abstraction

The sensor abstraction layer collects raw runtime signals and writes them into a synchronized data structure. The core contract between Layer 1 and Layer 2 is the `SignalFrame` struct. Each frame contains timestamped gaze, head, body, hand, spatial, and derived context fields, including timestamp, gaze origin and direction, fixation metadata, AOI hit, head position and forward vector, head-gaze divergence, posture class, spine angle, joint velocity, pinch states, interaction counts, nearest object distance, posture mode, boundary type, locomotion delta, context state, confidence, previous state, state hold duration, transition count, and nearest interactable reference.

The synchronizer emits a shared frame every 100 ms using `InvokeRepeating`, yielding a 10 Hz context-processing cadence in the current repository implementation.

4.2 Layer 2: Context Engine

The context engine transforms each synchronized frame into higher-level multimodal features, estimates a candidate context state through rule-based fusion, and stabilizes the result through a temporal state machine. This layer contains the main research logic of the prototype.

4.3 Layer 3: XR App Shell

The XR application layer consumes the current state and boundary context and selects the appropriate adaptive behavior. This layer is intentionally separated from the context engine so that inference logic remains independent of Unity-specific UI and scene objects.

5 Methodology

5.1 Task Environment

The application scene is a short sorting simulation implemented as `TrainingSimulation`. The user is presented with three task objects, a cube, cylinder, and sphere, which must be placed in the correct receptacles. Task objects and receptacles are tagged as Areas of Interest (AOIs), enabling the gaze subsystem to identify when task-relevant objects are being inspected. The scene is intentionally simple because the goal is not to study game mechanics but to isolate context inference during focused interaction.

5.2 Signal Acquisition

5.2.1 Gaze and AOI Detection

The current repository implementation resolves a gaze ray using the center-eye camera or center-eye anchor and casts the ray into the scene against AOI colliders. This

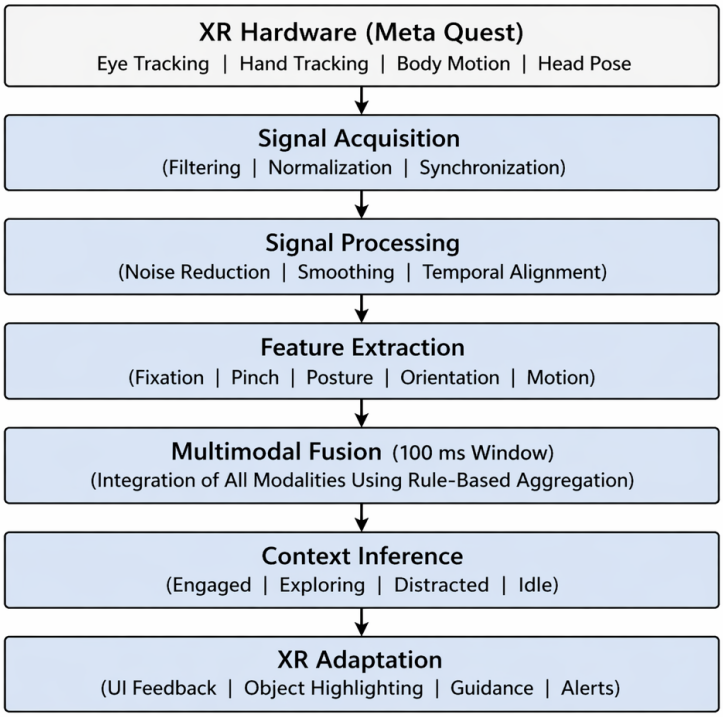

Figure 1: Real-Time Multimodal Context Inference Pipeline in XR

Figure 1: Reference pipeline figure from the earlier draft. The current repository still matches the overall multimodal processing flow, but the final paper text now maps it into the repository's three-layer structure: Sensor Abstraction, Context Engine, and XR App Shell.

produces gaze origin, normalized gaze direction, AOI hit name, a placeholder fixation estimate in Layer 1, and a rendered debug ray for QA. The code comments state that this update path runs in Unity `Update()` to follow headset refresh behavior at approximately 60–90 Hz.

5.2.2 Head and Body Signals

Head pose is captured from the main camera or OVR camera rig. Body pose capture uses `OVRSkeleton` joints and computes head position, head forward vector, head-gaze divergence, a coarse posture class, spine angle, and average joint velocity. The current Layer 1 posture labels are `standing`, `leaning`, `reaching`, and `unknown`, produced from spine-angle heuristics.

5.2.3 Hand Interaction Signals

Hand capture reads `OVRHand` input and controller fallback input. Pinch events are detected using a pinch-strength threshold of 0.7, and interaction counts are accumulated over a ten-second sliding window. The hand subsystem also computes the distance between the dominant tracked hand and the nearest AOI-tagged object.

5.2.4 Spatial Context Signals

Spatial context detection updates every five seconds and estimates `boundary_type` (`stationary`, `room_scale`, `custom`, or `passthrough`) and `posture_mode` (`sitting`, `standing`, or `room_scale`). Boundary type is inferred from the configured guardian geometry area. Posture mode is inferred from headset height relative to a session calibration point, with a sitting threshold difference of 0.25 m.

5.3 Signal Synchronization

The `SignalSynchroniser` collects the latest values from all capture scripts and writes them into a single `SignalFrame` every 100 ms. Missing values fall back to the most recent valid values or safe defaults. The current implementation does not yet expose an explicit per-signal staleness flag, which was requested in the implementation guide; this gap should be acknowledged as a limitation.

5.4 Feature Extraction

Feature extraction occurs in Layer 2 and converts raw frame values into inference-oriented features.

Gaze Features. `GazeFeatureExtractor` computes `currently_on_task_aoi`, `fixation_on_aoi`, `fixation_duration_s`, `head_gaze_divergence_deg`, `aoi_dwell_ratio`, and `saccade_rate_per_s`. The current implementation uses an I-VT velocity threshold of 30 deg/s, a minimum fixation duration of 80 ms [1], a 2000 ms saccade window, and a 5000 ms AOI dwell window.

Body Features. `BodyFeatureExtractor` maps raw posture and spine-angle values into `Upright`,

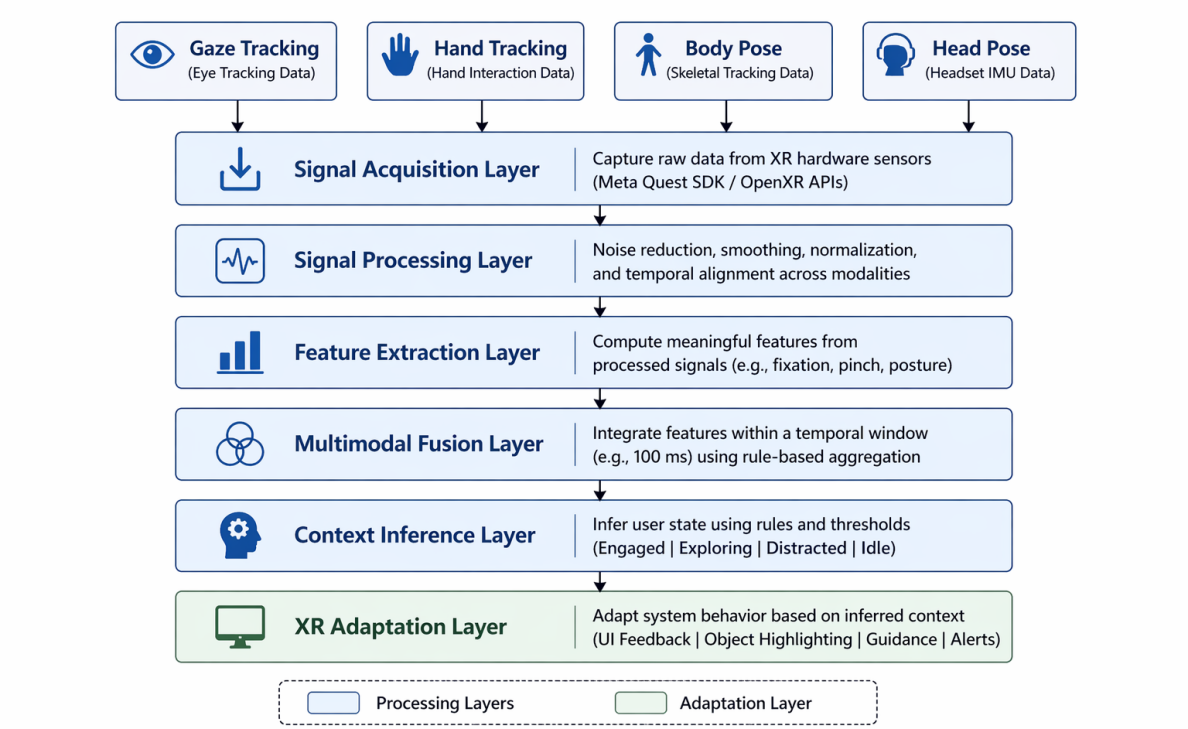

Figure 3: Modular Architecture for Multimodal Context Modeling in XR

Figure 2: Reference modular architecture figure from the earlier draft. The conceptual layering remains useful and is consistent with the current implementation, although the final paper uses the repository terminology of Sensor Abstraction, Context Engine, and XR Adaptation / App Shell.

`LeaningForward`, `Slouched`, and `LeaningBack`, and also passes forward average joint velocity.

Hand Features. `HandFeatureExtractor` computes interaction frequency as `interaction_count_10s / 10`, object proximity as `nearest_object_dist_m < 0.3`, and pinch activity as the logical OR of left and right pinch states.

Spatial Features. `SpatialContextExtractor` computes posture mode, boundary type, and locomotion activity. Locomotion is discretized from inter-frame head movement using the thresholds < 0.1 m (`None`), < 0.5 m (`Low`), and ≥ 0.5 m (`Active`).

5.5 Context State Definition

User context is inferred as one of four discrete behavioral states derived from multimodal feature integration. Each state represents a distinct level of attention, engagement, and interaction activity. States are determined by rule-based logic that evaluates integrated features within a temporal window, ensuring real-time classification and interpretability for downstream adaptive behaviors.

5.5.1 Engaged

Definition: User is actively interacting with task-relevant objects while maintaining focused visual attention and stable posture.

Measurable criteria: In the stronger reference formulation from the earlier draft, the Engaged state is associated with fixation on task-relevant AOIs, AOI dwell ratio above 0.6, interaction frequency above 0.1, posture within {Upright, LeaningForward}, and low head-gaze divergence. The current repository implementation uses a slightly more implementation-specific rule that combines task-AOI focus, fixation or active interaction, supportive posture, room-scale engagement cues, head-gaze divergence $\leq 18°$, and saccade rate $\leq 2.5/s$.

5.5.2 Distracted

Definition: User attention has deviated from task-relevant elements, interaction has reduced or ceased, and head orientation indicates attention drift.

Measurable criteria: In the stronger reference formulation, the Distracted state corresponds to no fixation on AOI, AOI dwell ratio below 0.3, head-gaze divergence above 20°, and minimal hand interaction. The current repository implementation uses a more concrete implementation rule based on not being on task AOI, no fixation on task AOI, AOI dwell below 0.20, away-signals from divergence or saccade activity or body movement, no active pinch, and low interaction frequency.

5.5.3 Transitioning

Definition: User is shifting attention between tasks or preparing to engage with a new target.

Measurable criteria: The reference draft defined this state using either head-gaze divergence above 15°

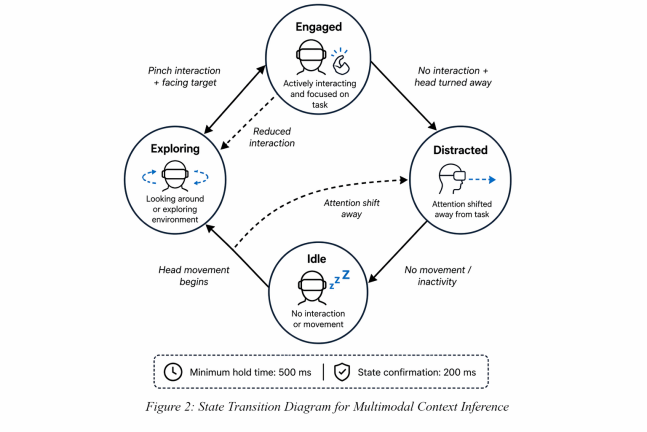

Figure 2: State Transition Diagram for Multimodal Context Inference

Figure 3: Reference state-transition diagram from the earlier draft. The older figure uses the label *Exploring*; in the current repository this behavior is represented by the implemented *Transitioning* state and related task-shift logic.

or combined posture change plus rising interaction. The current repository implementation captures a broader transitional pattern including high head-gaze divergence, task-adjacent but unstable gaze, short fixations, moderate AOI dwell, or preparatory hand proximity.

5.5.4 Idle

Definition: User has ceased all activity across all modalities and has maintained inactivity for a sustained duration.

Measurable criteria: The reference draft specifies average joint velocity below 0.05, zero interaction frequency, and no fixation on AOI for at least 5 seconds. The current repository implementation uses low body movement, no effective hand activity, no object proximity, no task-directed gaze, and very low AOI dwell.

5.6 Rule-Based Fusion and Hysteresis

The current `ContextFusionEngine` applies prioritized rules to infer a candidate state. Each inferred state receives a confidence score initialized from 0.5 and incremented based on supporting multimodal evidence, then clamped to the range $[0, 1]$. The `ContextStateMachine` reduces oscillation through a minimum state hold time of 0.5 s and a pending confirmation time of 0.2 s.

6 Implementation

6.1 Software Stack

The repository currently documents the following versions:

- Unity Editor: 2022.3.62f1
- OpenXR Plugin: 1.14.3
- Meta XR All-in-One SDK: 201.0.0
- Meta XR Core SDK: included with Meta XR All-in-One SDK
- OVRPlugin: 1.201.0

6.2 Runtime Integration

The `XRAppShellController` binds the context source to the application layer, publishes `XRContextSnapshot` updates, aligns the training simulation in front of the user, and provides fallback simulation behavior for QA. If the live context source cannot be found and fallback is enabled, the system can publish simulated states for manual testing.

6.3 Task Logic

The task layer uses `ReceptacleTrigger`, `SortableTaskObject`, and `TrainingSimulationTaskManager`. Correct placements are counted, and the task completes when all three objects are sorted.

6.4 Adaptive Behaviors

`AdaptationManager` maps current state and boundary type into adaptive behavior variants:

- Engaged: subtle vignette and reduced unnecessary hints
- Distracted: visual nudge, spatial audio cue, or minimal overlay depending on boundary mode
- Transitioning: next-task preload or content repositioning
- Idle: rest panel, continue / break / recenter controls, and optional floor arrow

6.5 Logging

`ContextLogger` writes each processed frame to a timestamped `.jsonl` file under `Application.persistentDataPath`. Logged entries include full signal-frame payload, context state, confidence, previous state, hold duration, state-transition count, and nearest interactable reference.

7 Evaluation Design

This section reflects the required empirical paper design from the implementation guide and WBS. Replace placeholders with the real study details before submission.

7.1 Study Design

The planned study is a within-subjects comparison with two conditions:

- **Baseline condition:** system active, adaptive behaviors disabled
- **Adaptive condition:** full multimodal inference and adaptive behaviors enabled

Condition order should be counterbalanced across participants.

7.2 Participants

Target participants: 8–10. Minimum usable sample for inferential analysis: 6.

7.3 Procedure

Each participant session is planned as follows:

1. Consent and briefing
2. Familiarization with headset and sorting task
3. Condition A or B (8–12 minutes)
4. NASA-TLX and self-report ratings
5. Five-minute break
6. Remaining condition
7. NASA-TLX and self-report ratings
8. Debrief

7.4 Measures and Analysis

The primary measure is context detection accuracy against participant self-report or ground-truth labeling. Secondary measures are task completion time, number of sorting errors, NASA-TLX workload score, and subjective engagement and distraction ratings. The implementation guide specifies descriptive summaries, a confusion matrix for state classification, and a Wilcoxon signed-rank test for Adaptive vs Baseline comparisons.

8 Current Project Status for the Paper

Considering the old draft only as reference, the current project is no longer just a concept draft. The repository now contains a real implemented pipeline for the paper's core technical claim: signal schema, synchronization, feature extraction, rule-based fusion, state hysteresis, XR adaptation, task progression, and JSONL logging are all present in code. Therefore, the paper can now strongly support the architecture, methodology, and implementation sections.

What remains incomplete for a fully submission-ready empirical paper is the evidence layer: the final user-study results, the complete participant and ethics details, and the full verified related-work bibliography from the older draft. In other words, the system side is substantially stronger than before, while the evaluation side still needs completion.

9 Discussion

The intended interpretation of the system is that multimodal sensing provides a more meaningful representation of user context than isolated XR signals. The design of the prototype supports this claim at the system-architecture level because it combines attention, posture, interaction, and spatial setup before selecting adaptive support. The adaptive layer is deliberately lightweight: the goal is not to interrupt the user but to provide support that matches inferred state.

From an HCI perspective, the most important design principle in the current prototype is not the specific threshold values but the explicit separation between sensing, inference, and application behavior. This separation improves inspectability, modularity, and future extensibility. It also creates a path toward a plugin-style architecture in which the context engine can later be replaced by a learned model without rewriting the XR application layer.

10 Limitations

Several limitations should be stated explicitly in the final paper. First, the current repository implementation is still a proof-of-concept. Second, the current task is deliberately simple, so ecological validity for richer training workflows remains limited. Third, the fusion model is manually authored and threshold-driven, which improves interpretability but may reduce generalization. Fourth, the current repository does not yet include explicit per-signal staleness flags, even though the implementation guide proposed them. Fifth, the current gaze implementation in the repository is a center-eye gaze-ray approximation rather than a fully documented hardware eye-tracking pipeline; the paper should not overclaim beyond the implemented code path. Finally, the complete bibliography from the older draft and the final empirical results are not yet available in machine-readable form, so those elements still require restoration.

11 Future Work

The planning documents identify several extensions that should be preserved in the final manuscript: locomotion-aware context refinement, face-tracking integration, EEG or physiological augmentation, multi-user or shared-space support, and personalization of thresholds and adaptation policies. An important near-term next step is the creation of a labeled session dataset from the current logging pipeline. That dataset would support direct comparison between the current rule-based engine and future learned classifiers.

12 Conclusion

GazePose-Context presents a structured XR prototype for multimodal context inference and adaptive training support. The system combines gaze, head, body, hand, and spatial context signals inside a three-layer Unity architecture and maps the resulting inferred states to lightweight, boundary-aware adaptive behavior. The repository already supports synchronized signal capture, feature extraction, rule-based fusion, state stabilization, runtime adaptation, and persistent logging. This LaTeX paper version updates the manuscript into paper-ready format, integrates the current implementation status and available reference materials, and is ready for

further refinement once the remaining verified references and final study results are supplied.

Acknowledgments

This research used AI-assisted tools in the following capacities: Claude for paper drafting assistance and research synthesis; GitHub Copilot for Unity C# code scaffolding, with all generated code tested and validated on hardware by the researchers; literature-discovery tools for reference discovery; transcription tools for interview transcription if applicable; and charting tools for results figure generation. All scientific claims, research contribution statements, experimental results, and final citations were produced and verified by the human authors.

References

[1] D. D. Salvucci and J. H. Goldberg, "Identifying fixations and saccades in eye-tracking protocols," *Proceedings of the 2000 Symposium on Eye Tracking Research & Applications*, pp. 71–78, 2000. doi: https://doi.org/10.1145/355017.355028

[2] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," in *Advances in Psychology*, vol. 52, P. A. Hancock and N. Meshkati, Eds. Amsterdam, The Netherlands: North-Holland, 1988, pp. 139–183.